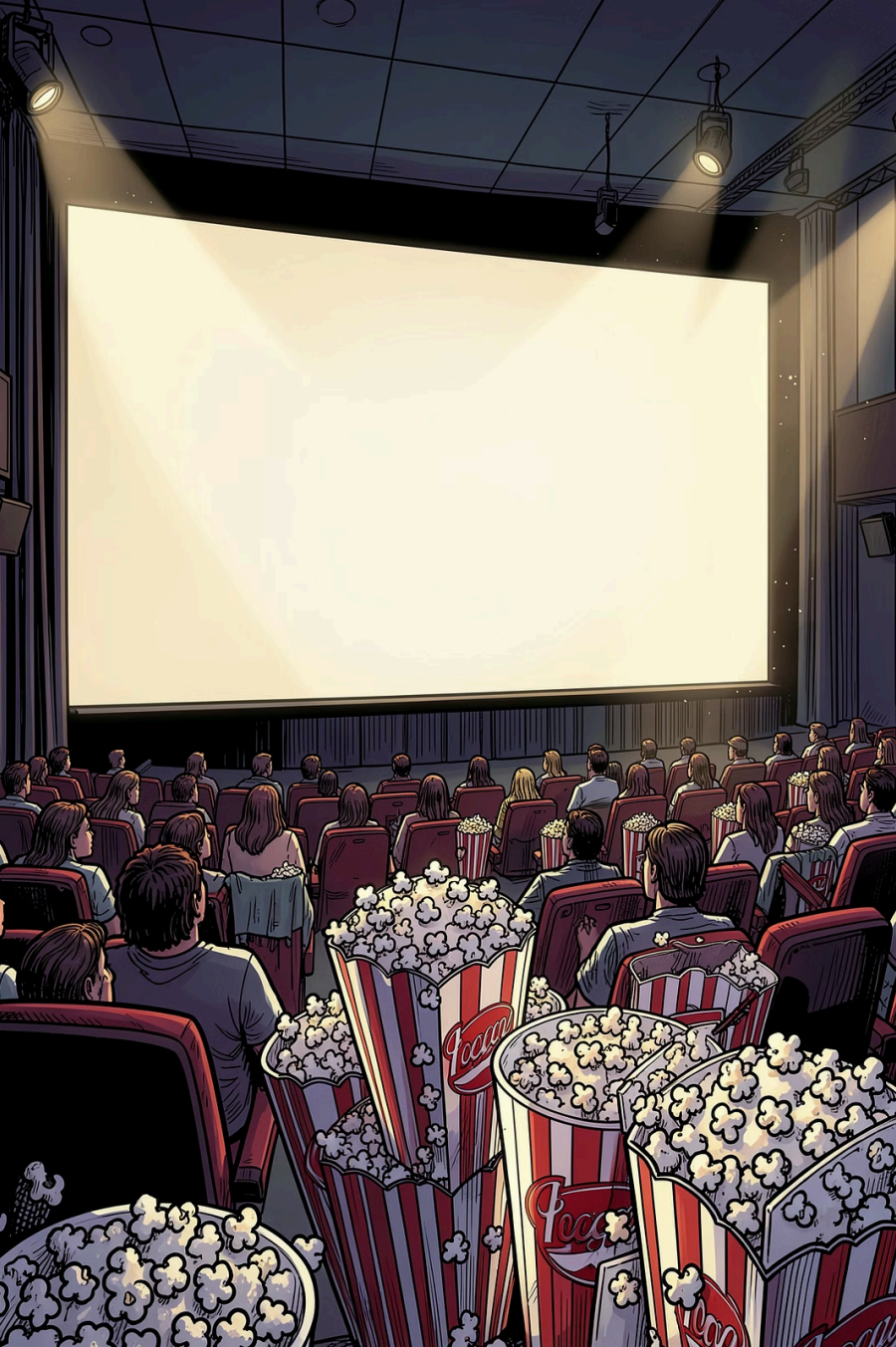




Movie Review Sentiment Analysis

IMDB Dataset — Machine Learning Classification Report using Python, Scikit-learn, NLTK, and XGBoost.

50,000 REVIEWS

BINARY CLASSIFICATION

POSITIVE / NEGATIVE

Problem Statement

With the rapid rise of user-generated content on platforms like IMDb and Rotten Tomatoes, automatically classifying movie reviews as **Positive** or **Negative** has become essential for companies and consumers.

Dataset

50,000 labeled IMDB movie reviews

Task

Binary sentiment classification from text

Goal

Compare 6 ML models to find the best performer



Dataset Overview

Index	Rating	A	B	Index	D
1	2.7	Movie Review /M			
4	1.0	Movie Review	Movie Review		
6	2.7	Movie Review	Movie Review		
4	1.9	Movie Review /M	Movie Review		
3	1.6	Movie Review	Movie Review		
2	1.2	Movie Review	Movie Review		
6	1.8	Movie Review			
☒	2.3	Movie Review			
☒	3.5	Movie Review	Movie Review		
☒	1.0	Movie Review	Movie Review		
☒	2.3	Movie Review	Movie Review		
☒	2.7	Movie Review			



IMDB Movie Review Dataset

A widely-used benchmark for sentiment analysis tasks.

Total Rows	50,000
Columns	2 (review, sentiment)
Class Distribution	25K Positive / 25K Negative
Null Values	0
Duplicates Removed	418

i The dataset is **perfectly balanced** (50/50), eliminating the need for oversampling or undersampling. Reviews contained HTML tags, URLs, punctuation, and slang requiring cleanup.

Data Preprocessing Pipeline

Raw text cannot be fed directly into ML models. A thorough pipeline was applied to clean and normalize reviews before feature extraction.

01

Text Normalization

Lowercasing, HTML tag removal, URL stripping, punctuation removal

02

Semantic Preservation

Slang conversion (80+ terms), emoji-to-text via `emoji.demojize()`, negation words kept (e.g., "not", "never")

03

Tokenization & Stemming

NLTK `word_tokenize()` + Porter Stemmer (e.g., "running" → "run")

Feature Engineering

TF-IDF Vectorization

Primary feature extraction method — weighs words by frequency in a document relative to all documents.

max_features	10,000
ngram_range	(1, 2) — captures bigrams
min_df / max_df	2 / 0.95
sublinear_tf	True

Additional Steps

Label Encoding

Negative = 0, Positive = 1

Bag of Words

CountVectorizer with max_features=5,000

Train-Test Split

80% training (~39K) / 20% testing (~9,917)

Six Machine Learning Models

Each model represents a different algorithmic approach to classification.



Logistic Regression

$C=4$, $\text{max_iter}=2000$ — linear model, highly effective for high-dimensional text



Naive Bayes

$\alpha=0.3$ — probabilistic, fast, works well with text



Decision Tree

$\text{max_depth}=100$ — interpretable but prone to overfitting



Random Forest

$n_estimators=200$ — ensemble, reduces overfitting



SVM (LinearSVC)

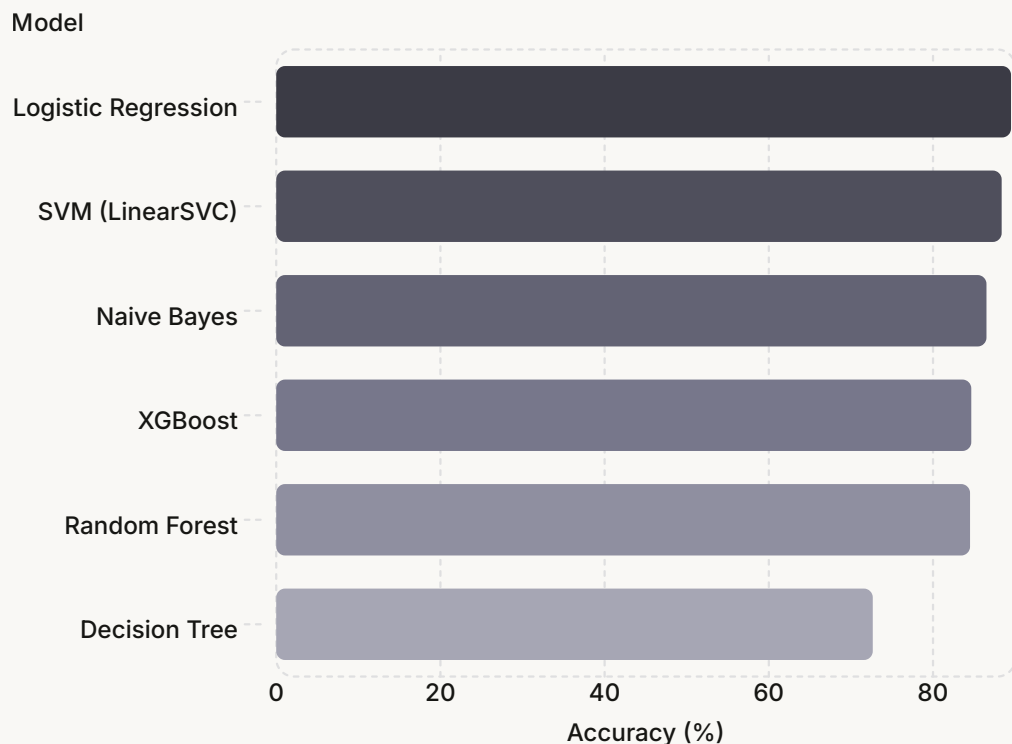
$C=1.5$ — optimal hyperplane, excellent for sparse text



XGBoost

$n_estimators=200$ — gradient boosting, powerful on structured data

Model Accuracy Comparison



Key Takeaways

Logistic Regression leads at **89.50%**, followed closely by SVM at **88.36%**. Linear models significantly outperform tree-based methods on sparse TF-IDF features.

❏ **Decision Tree** had the lowest accuracy (72.66%), struggling with high-dimensional feature space.

Top Performers: Logistic Regression & SVM

Logistic Regression — 89.50%

Best overall performer with balanced precision and recall across both classes.

- Only 568 negative reviews misclassified as positive
- Only 473 positive reviews misclassified as negative
- F1-Score: 0.90 (both classes)

SVM (LinearSVC) — 88.36%

Second place, highly suitable for high-dimensional sparse TF-IDF spaces.

- 613 negative reviews misclassified as positive
- 541 positive reviews misclassified as negative
- F1-Score: 0.88 (both classes)

Other Models at a Glance



Naive Bayes — 86.51%

Fast and simple. Misclassified 754 negative reviews as positive — solid despite independence assumption rarely holding in real text.



Random Forest — 84.51%

Improved 12% over Decision Tree via ensemble approach. Computationally expensive; still behind linear models.



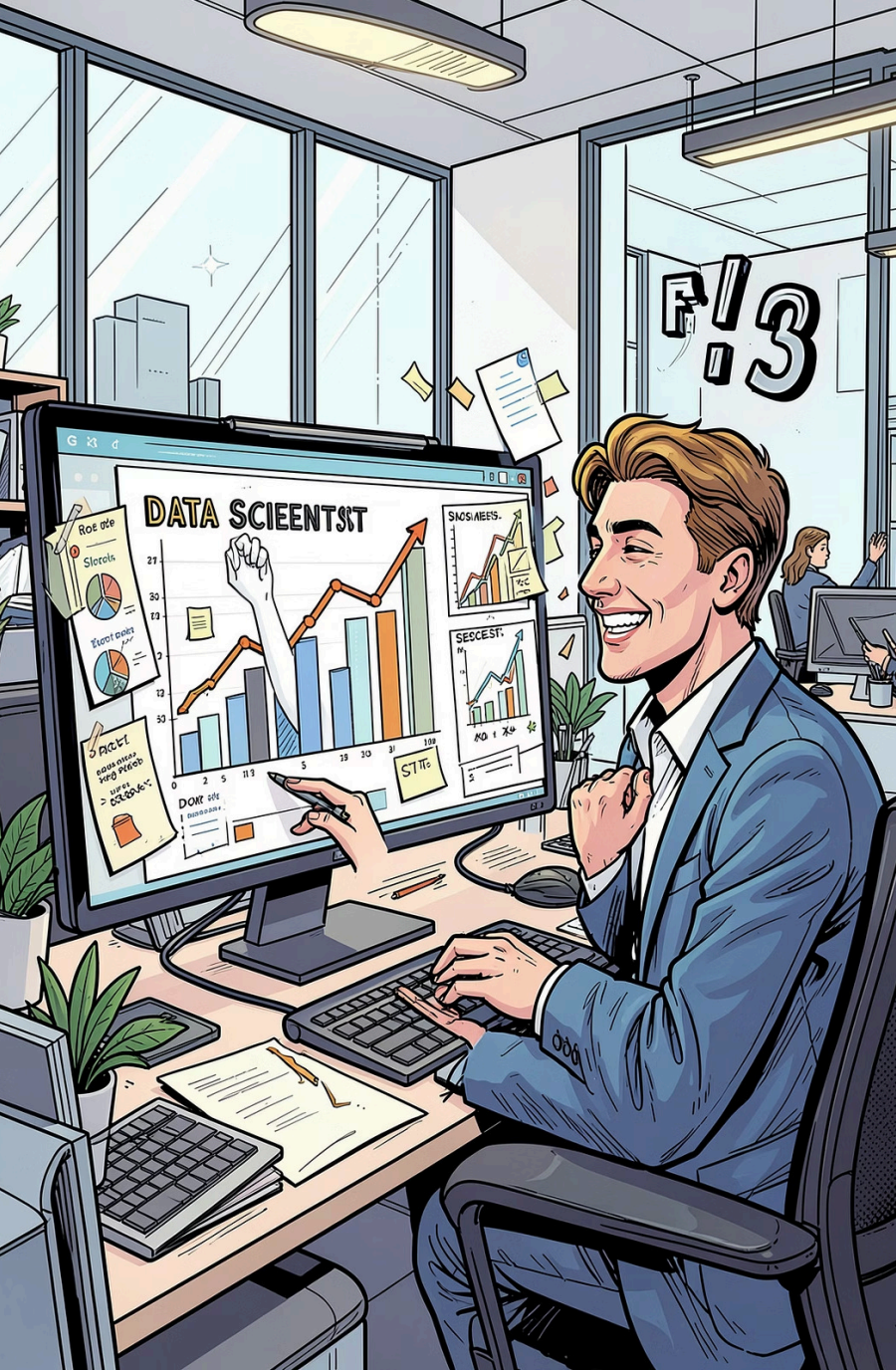
XGBoost — 84.66%

Similar to Random Forest. More effective on structured/tabular data than sparse TF-IDF text features.



Decision Tree — 72.66%

Lowest accuracy. High misclassifications (1,307 + 1,404) confirm it is not well-suited for high-dimensional TF-IDF features.



Conclusion & Recommendation

This project built an end-to-end sentiment analysis pipeline on 50,000 IMDB reviews using six ML models.



Recommended Model

Logistic Regression —
89.50% accuracy, balanced precision-recall, fast inference, and interpretable coefficients

Key Preprocessing Wins

Keeping negation words + using bigrams (ngram_range=(1,2)) preserved critical sentiment signals like "not good"

Linear Models Dominate

Logistic Regression & SVM outperform tree-based methods on sparse TF-IDF text features